

State of Charge Dual Estimation of a Li-ion Battery Based on Variable Forgetting Factor Recursive Least Square and Multi-Innovation Unscented Kalman Filter Algorithm

Sistem Kendali Prediktif Adaptif



Muhammad Fikri
Rizki Abyadhi
2106640146



Dicky Osmond
Sieliewangi
2206026271

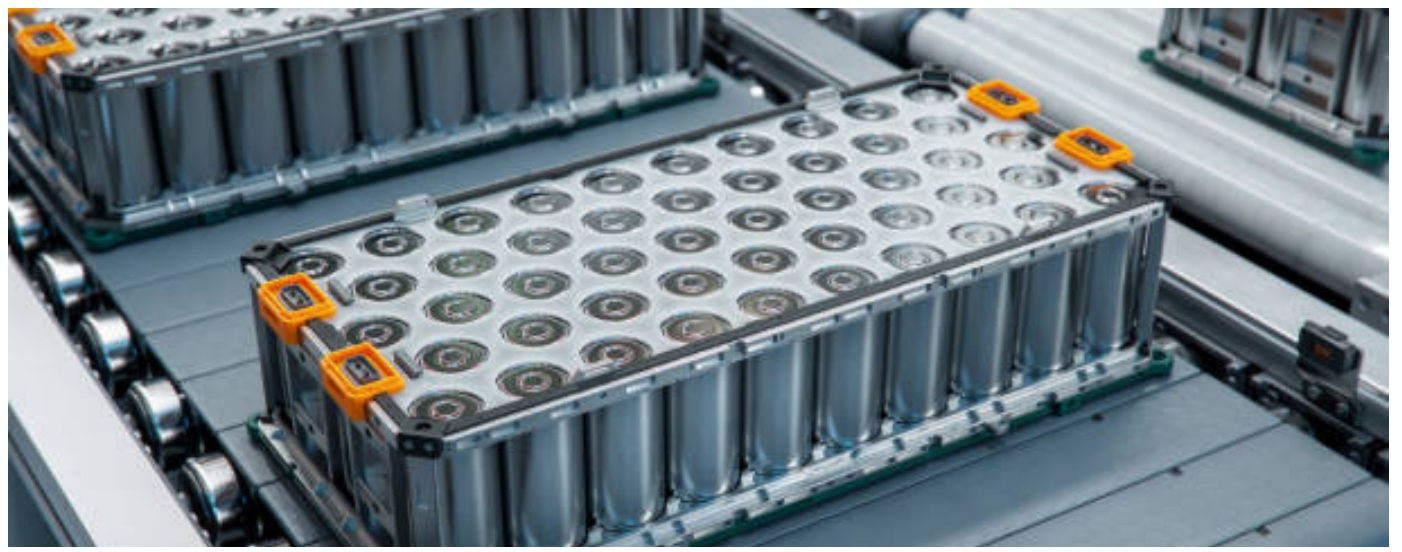


Muhammad
Farhan Hanafi
2206028062

Latar Belakang

Sistem Manajemen Baterai (Battery Management System/BMS) merupakan komponen yang sangat penting dalam kendaraan listrik karena berperan langsung dalam **menjaga keamanan, efisiensi, dan memperpanjang umur pakai baterai**.

Salah satu fungsi paling krusial dari BMS adalah melakukan **estimasi State of Charge (SOC) secara akurat dan real-time**. Estimasi nilai SOC yang akurat sangat penting, karena digunakan untuk menentukan kapan baterai harus diisi ulang atau dihentikan dari pengisian, serta sebagai acuan bagi sistem manajemen energi untuk mengambil keputusan yang optimal dalam pengoperasian kendaraan listrik.



Masalah Utama dalam Estimasi SOC

- **Sifat SOC yang Tidak Bisa Diukur Langsung**, Perlu pendekatan estimasi berbasis model atau algoritma untuk memperkirakan nilainya.
- **Karakteristik Nonlinier dan Dinamis dari SOC**, SOC dipengaruhi oleh banyak sekali faktor, seperti Reaksi kimia internal baterai, Suhu lingkungan, Beban arus yang berubah-ubah, serta Usia dan degradasi baterai.
- **Keterbatasan Metode Estimasi Tradisional**, Misalnya metode coulomb counting dan Open Circuit Voltage, yang rentan terhadap akumulasi error
- **Keterbatasan Kalman Filter (KF) Klasik**, KF mengasumsikan sistem linier, Padahal, sistem baterai sangat nonlinier dan memiliki memori historis

Tujuan Penelitian

- **Mengembangkan Algoritma Dual Estimation SOC**, VFFRLS untuk mengestimasi parameter model baterai secara online dan adaptif. MIUKF untuk mengestimasi nilai SOC berdasarkan pengukuran terminal voltage dan input arus.
- **Memanfaatkan Keunggulan Pendekatan Dual**, Parameter baterai tidak diasumsikan tetap, tetapi diperbarui secara real-time dan mengatasi keterbatasan UKF klasik dengan memasukkan informasi kesalahan historis ke dalam proses update.

Konsep Umum

Salah satu metode paling dasar dan banyak digunakan dalam sistem manajemen baterai adalah Ampere-Hour Integration, atau yang lebih dikenal sebagai Coulomb Counting. Metode ini menghitung SOC dengan **mengintegrasikan arus yang mengalir keluar dan masuk ke dalam baterai dari waktu ke waktu**, berdasarkan prinsip bahwa jumlah muatan listrik total (dalam satuan Coulomb atau Ah) yang masuk dan keluar dapat dipakai untuk menilai kapasitas baterai yang tersisa.

Persamaan Umum

$$SOC(t) = SOC_0 - \frac{1}{Q_N} \int_{t_0}^t \mu \cdot I(t) dt$$

- SOC_0 : nilai awal State of Charge (SOC) pada waktu t_0
- Q_N : kapasitas nominal baterai (dalam Ah atau Coulomb)
- $I(t)$: arus pada waktu t (positif saat discharge, negatif saat charge)
- μ : efisiensi coulombic, biasanya bernilai mendekati 1

Keunggulan Metode

- **Implementasi sederhana:** hanya memerlukan sensor arus dan integrator numerik.
- **Waktu-nyata (Real-time):** dapat langsung menghitung SOC selama pengoperasian baterai.
- Respon cepat terhadap perubahan arus.

Kelemahan Metode

- **Ketergantungan pada nilai awal SOC (initial SOC):** Jika nilai awal tidak akurat, seluruh estimasi SOC selanjutnya akan salah.
- **Akumulasi error akibat kesalahan sensor:** Sensor arus memiliki noise dan offset, yang menyebabkan drift estimasi SOC.
- **Sensitivitas terhadap efisiensi Coulomb (μ):** Jika efisiensi diset terlalu ideal ($\mu = 1$), maka rugi energi atau konversi tidak dipertimbangkan

Kalman Filter Klasik

KF bekerja berdasarkan prinsip filtering rekursif Bayesian, di mana estimasi status sistem diperbarui secara berkelanjutan menggunakan data dari dua dimensi informasi yang relatif independen: prediksi model dan hasil pengukuran aktual. Secara umum, algoritma Kalman Filter didesain **untuk sistem yang linier dengan gangguan yang bersifat white Gaussian noise**.

Persamaan Umum KF

$$x_{k+1} = A_{k+1}x_k + B_{k+1}u_{k+1} + \omega_{k+1}$$
$$y_{k+1} = H_{k+1}x_{k+1} + \nu_{k+1}$$

x_k : vektor status sistem pada waktu ke- k
 u_k : input sistem (misal arus baterai)
 y_k : vektor observasi (misal tegangan terminal)
 A_k, B_k : matriks transisi status dan input kontrol
 H_k : matriks observasi
 ω_k, ν_k : white noise pada sistem dan pengukuran, masing-masing
 $\omega_k \sim \mathcal{N}(0, Q_k), \quad \nu_k \sim \mathcal{N}(0, R_k)$

Extended Kalman Filter

EKF mengatasi nonlinieritas dengan cara melinearisasi fungsi nonlinier pada model status dan pengukuran **menggunakan ekspansi deret Taylor orde pertama**. Fungsi nonlinier seperti $f(x, u)$ dan $h(x)$ disederhanakan menggunakan turunan parsial (Jacobian), sehingga KF tetap bisa diterapkan. Namun, karena proses linearitas hanya sampai orde pertama, EKF menjadi sensitif terhadap dinamika yang sangat nonlinier.

Unscented Kalman Filter

Sebagai solusi terhadap kekurangan EKF, dikembangkanlah UKF, yang mengusung prinsip bahwa lebih mudah untuk melakukan aproksimasi distribusi probabilitas daripada fungsi nonliniernya. UKF menggunakan teknik unscented transformation untuk **membangkitkan sejumlah titik sigma di sekitar estimasi status**, kemudian menyebarkan titik-titik ini melalui fungsi nonlinier tanpa perlu linearisasi.

Menghitung Sigma Points

$$\begin{cases} \chi_k^0 = x_k^{(+)} \\ \chi_k^i = x_k^{(+)} + \sqrt{(L + \eta)P_{xx}}, i = 1, 2 \dots L \\ \chi_k^i = x_k^{(+)} - \sqrt{(L + \eta)P_{xx}}, i = L + 1, L + 2, \dots 2L \end{cases}$$
$$\begin{cases} \eta = \alpha^2(L + k_i) - L \\ W_m^0 = \frac{\eta}{L + \eta}, W_m^i = \frac{1}{2(L + \eta)}, i = 1, 2 \dots 2L \\ W_c^0 = \frac{\eta}{L + \eta} + 1 - \alpha^2 + \beta, W_c^i = \frac{1}{2(L + \eta)}, i = 1, 2 \dots 2L \end{cases}$$

Update A Priori

$$\chi_{k+1}^i = f(\chi_k^i, u_k, k)$$
$$x_{k+1}^{(-)} = \sum_{i=0}^{2L} W_m^i \chi_{k+1}^i$$
$$P_{k+1}^{(-)} = \sum_{i=0}^{2L} (W_c^i (\chi_k^i - x_{k+1}^{(-)}) (\chi_k^i - x_{k+1}^{(-)})^T) + Q_{k+1}$$

Update posteriori

$$K_{k+1} = \frac{P_{k+1}^{xy}}{P_{k+1}^{yy}}$$
$$x_{k+1}^{(+)} = x_{k+1}^{(-)} + K_{k+1}(y_{k+1} - \hat{y}_{k+1}) \quad P_{k+1}^{(+)} = P_{k+1}^{(-)} - K_{k+1}P_{k+1}^{yy}K_{k+1}$$

Menghitung Estimasi Observasi

$$y_{k+1}^i = h(\chi_{k+1}^i, k + 1)$$
$$\hat{y}_{k+1} = \sum_{i=0}^{2L} W_m^i y_{k+1}^i$$
$$P_{k+1}^{yy} = \sum_{i=0}^{2L} (W_c^i (y_{k+1}^i - \hat{y}_{k+1}) (y_{k+1}^i - \hat{y}_{k+1})^T) + R_{k+1}$$
$$P_{k+1}^{xy} = \sum_{i=0}^{2L} W_c^i (\chi_{k+1}^i - \hat{x}_{k+1}^{(-)}) (y_{k+1}^i - \hat{y}_{k+1})^T$$

Multi-Innovation UKF

Untuk mengatasi batasan UKF klasik, paper ini mengusulkan pengembangan lebih lanjut bernama Multi-Innovation Unscented Kalman Filter (MIUKF). MIUKF **menambahkan kontribusi dari beberapa error historis (innovations) dalam proses koreksi status**, bukan hanya error di saat ini.

Persamaan Tambahan Pada MIUKF

Vektor Error

$$e_{p,k} = \begin{bmatrix} e_k \\ e_{k-1} \\ e_{k-2} \\ \vdots \\ e_{k-p+1} \end{bmatrix}$$

Koefisien Bobot

$$\begin{cases} \gamma_1 = 1 \\ \gamma_2 = \gamma_3 = \dots \gamma_p = \frac{a}{M-1}, 0 \leq a \leq 1, M \geq 2 \end{cases}$$

Kalman gain

$$k_{p,k} = [k_k, k_{k-1}, \dots, k_{k-p+1}]$$

Update State

$$y_k = \hat{y}_k + \sum_{i=0}^p \gamma_i k_{i,k} e_{k-i}$$

- e_k : inovasi (error) pada waktu ke- k , $e_k = y_k - \hat{y}_k$
- $e_{p,k}$: vektor inovasi historis sepanjang p langkah waktu
- k_k : Kalman gain pada waktu ke- k
- $k_{p,k}$: matriks Kalman gain historis: $[k_k, k_{k-1}, \dots, k_{k-p+1}]$
- $\gamma_0 = 1, \quad \gamma_i = \frac{a}{M-1}$ untuk $i = 1, 2, \dots, p$
- $a \in [0, 1]$ adalah bobot penyetelan kontribusi historis
- $M \geq 2$ adalah panjang jendela inovasi (window length)
- y_k : status sistem setelah koreksi inovasi multi-waktu
- $\hat{y}_k$: prediksi sistem dari UKF sebelum koreksi
- $\gamma_0 = 1$ (inovasi saat ini diberi bobot penuh)
- $\gamma_1 = \gamma_2 = \dots = \gamma_p = \frac{a}{M-1}$ (inovasi historis berbagi bobot merata)
- a : koefisien kontrol pengaruh inovasi masa lalu, $0 \leq a \leq 1$
- M : panjang jendela inovasi (minimal 2)
- $p = M - 1$ (jumlah langkah historis yang digunakan)



Pendekatan ini lebih realistis untuk sistem seperti baterai, yang statusnya dipengaruhi oleh dinamika masa lalu, suhu, beban siklikal, dan degradasi internal.



Dengan demikian, transisi dari KF → EKF → UKF → MIUKF mencerminkan evolusi algoritma estimasi SOC menuju akurasi yang lebih tinggi dan ketahanan terhadap sistem nonlinier kompleks.

Pendahuluan

Secara umum, identifikasi parameter terbagi menjadi dua kategori utama: offline dan online. Pada metode offline, parameter diestimasi melalui data uji laboratorium dalam kondisi tertentu, biasanya menggunakan fitting model berdasarkan respons impuls atau siklus pengisian/discharging. Meskipun akurat dalam kondisi terbatas, **parameter offline bersifat statis dan tidak bisa beradaptasi terhadap perubahan operasional baterai yang dinamis**. Sebaliknya, pendekatan online identification memungkinkan parameter diperbarui secara waktu nyata, mengikuti perubahan karakteristik baterai saat digunakan.

Recursive Least square

Salah satu metode online paling umum digunakan untuk tujuan ini adalah Recursive Least Squares (RLS). Metode RLS sangat efisien secara komputasi dan ideal untuk sistem real-time karena memperbarui estimasi parameter secara rekursif dari data input-output yang diamati. RLS bekerja dengan meminimalkan fungsi kesalahan kuadrat secara inkremental tanpa perlu menyimpan seluruh data historis.

Namun, metode RLS biasa memiliki kelemahan pada faktor pelupa yang nilainya statis

$$\theta_k = \theta_{k-1} + K_k (y_k - \varphi_k^\top \theta_{k-1})$$

$$K_k = \frac{P_{k-1} \varphi_k}{\lambda + \varphi_k^\top P_{k-1} \varphi_k}$$

$$P_k = \frac{1}{\lambda} (P_{k-1} - K_k \varphi_k^\top P_{k-1})$$

- θ_k : vektor parameter model pada waktu ke- k
- φ_k : vektor regressor (input sistem)
- y_k : output sistem (misalnya tegangan terminal)
- K_k : gain RLS (ukuran seberapa besar koreksi)
- P_k : matriks kovarians estimasi
- λ : faktor pelupa (forgetting factor), $0 < \lambda \leq 1$

Variable Forgetting Factor-RLS

Untuk mengatasi keterbatasan RLS biasa, dikembangkanlah algoritma Variable Forgetting Factor Recursive Least Squares (VFFRLS), yang secara dinamis menyesuaikan nilai λ berdasarkan error estimasi saat ini. **VFFRLS memungkinkan sistem untuk lebih responsif saat terjadi perubahan signifikan dalam parameter**, dan tetap stabil saat sistem berada dalam keadaan steady-state.

$$\lambda_k = 1 - \alpha \left(\frac{e_k^\top e_k}{\varphi_k^\top P_{k-1} \varphi_k} \right)$$

λ_k : faktor pelupa adaptif pada waktu ke- k

$e_k = y_k - \varphi_k^\top \theta_{k-1}$: error estimasi

α : parameter penyesuaian sensitivitas forgetting factor

φ_k, P_{k-1} : vektor regressor dan matriks kovarians sebelumnya

Metode Levenberg–Marquardt

Algoritma Levenberg–Marquardt (LM) adalah salah satu metode numerik yang sangat populer untuk menyelesaikan masalah optimasi nonlinear berbasis regresi, terutama ketika model matematis bersifat nonlinear terhadap parameternya. LM **menggabungkan kekuatan dari dua metode: metode Gauss–Newton (cepat saat mendekati solusi) dan gradient descent (stabil saat jauh dari solusi).**

LM digunakan untuk mencari parameter optimal $\mathbf{x}$ yang meminimalkan fungsi loss kuadrat dari bentuk:

$$\min_{\mathbf{x}} \|\mathbf{r}(\mathbf{x})\|^2 = \sum_{i=1}^m [r_i(\mathbf{x})]^2$$

Di mana $r(x)$ adalah residual function, yaitu selisih antara nilai target yaitu koefisien diskrit a_1, a_2, b_0, b_1, b_2 dengan nilai hasil perhitungan dari model saat ini berdasarkan R_0, R_1, R_2, C_1, C_2

Langkah Iteratif Levenberg Marquardt

Untuk menemukan parameter optimal $\mathbf{x}^*$, LM memperbarui nilai parameter di setiap iterasi dengan rumus:

$$(J^T J + \mu I) \Delta x = -J^T \mathbf{r} \Rightarrow \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta x$$

J : Jacobian matrix, yaitu turunan parsial dari residual terhadap parameter $\mathbf{x}$, berukuran $m \times n$

$\mathbf{r}$: Vektor residual, yaitu $r_i = y_i^{\text{target}} - y_i^{\text{model}}(\mathbf{x})$

μ : Parameter damping untuk mengatur transisi antara Gauss–Newton dan Gradient Descent

I : Matriks identitas berukuran $n \times n$

Δx : Vektor koreksi parameter yang ditambahkan ke $\mathbf{x}_k$

$\mathbf{x}_k$: Nilai parameter pada iterasi ke- k

$\mathbf{x}_{k+1}$: Nilai parameter setelah update, yaitu $\mathbf{x}_k + \Delta x$

Dasar Teori

Model Listrik Baterai

Deskripsi Umum

Paper ini menggunakan struktur Equivalent Circuit Model bertipe **Dual Polarization** (DP) Model (orde-2), yang dianggap lebih akurat dibanding model RC orde-1 (single polarization) karena dapat merepresentasikan dua jenis efek polarisasi:

1. Polarisasi elektrokimia (slow dynamics),
2. Polarisasi ohmik (fast dynamics).

State Model

$$\frac{dU_1}{dt} = -\frac{1}{R_1 C_1} U_1 + \frac{1}{C_1} I(t)$$

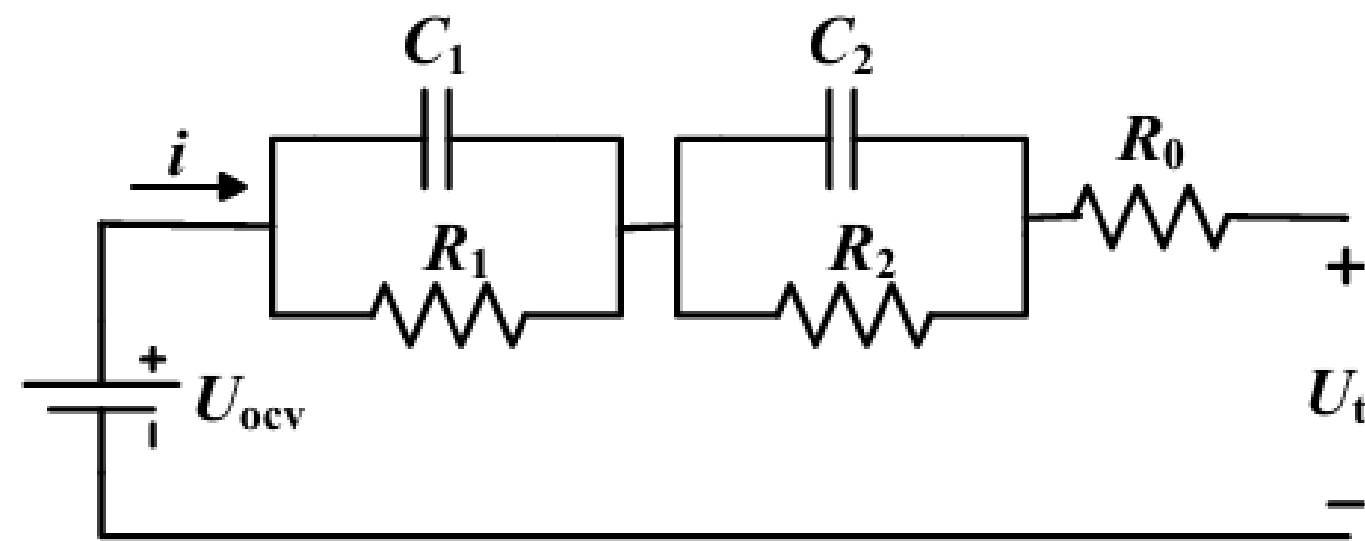
$$\frac{dU_2}{dt} = -\frac{1}{R_2 C_2} U_2 + \frac{1}{C_2} I(t)$$

Output Model

$$U_t(t) = U_{oc}(SOC(t)) - U_1(t) - U_2(t) - R_0 \cdot I(t)$$

$U_t(t)$: tegangan terminal baterai
 $U_{oc}(SOC)$: tegangan open-circuit sebagai fungsi SOC
 U_1, U_2 : tegangan pada cabang RC pertama dan kedua
 R_0 : resistansi internal ohmik
 R_1, R_2 : resistansi pada RC branch 1 dan 2
 C_1, C_2 : kapasitansi pada RC branch 1 dan 2
 $I(t)$: arus baterai pada waktu t
 Q_n : kapasitas nominal baterai

Equivalent Circuit Model



➤ Struktur DP Model terdiri dari komponen:

- R_0 : resistansi internal ohmik baterai, yang langsung memengaruhi tegangan terminal saat arus diterapkan.
- R_1, C_1 : RC branch pertama untuk memodelkan polarisasi permukaan (surface polarization).
- R_2, C_2 : RC branch kedua untuk memodelkan polarisasi difusi (diffusion polarization).
- $U_{oc}(SOC)$: tegangan open-circuit baterai sebagai fungsi nonlinier dari SOC.

$$U_t = U_{ocv} - I R_0 - U_1 - U_2,$$

$$\dot{U}_1 = -\frac{1}{C_1 R_1} U_1 + \frac{1}{C_1} I,$$

$$\dot{U}_2 = -\frac{1}{C_2 R_2} U_2 + \frac{1}{C_2} I.$$

$$H(s) = \frac{U_{ocv}(s) - U_4(s)}{I(s)} = \frac{1}{C_1 \left(s + \frac{1}{C_1 R_1} \right)} + \frac{1}{C_2 \left(s + \frac{1}{C_2 R_2} \right)} + R_0,$$

Dengan Tustin $S = \frac{2}{T} \frac{z-1}{z+1}$

$$\frac{U_{ocv}(z) - U_4(z)}{I(z)} = \frac{1}{C_1 \left(\frac{2}{T} \frac{z-1}{z+1} + \frac{1}{C_1 R_1} \right)} + \frac{1}{C_2 \left(\frac{2}{T} \frac{z-1}{z+1} + \frac{1}{C_2 R_2} \right)} + R_0.$$

Nominator

$$\begin{aligned} N(z) = & \left[T(R_1(2C_2 R_2 + T) + R_2(2C_1 R_1 + T)) + R_0(2C_1 R_1 + T)(2C_2 R_2 + T) \right] z^0 \\ & + \left[T(R_1(2C_2 R_2 + T) + R_2(2C_1 R_1 + T)) + T(R_1(T - 2C_2 R_2) + R_2(T - 2C_1 R_1)) \right. \\ & + R_0[(2C_1 R_1 + T)(T - 2C_2 R_2) + (2C_2 R_2 + T)(T - 2C_1 R_1)] \left. \right] z^{-1} \\ & + \left[T(R_1(T - 2C_2 R_2) + R_2(T - 2C_1 R_1)) + R_0(T - 2C_1 R_1)(T - 2C_2 R_2) \right] z^{-2}. \end{aligned}$$

Denominator

$$\begin{aligned} D(z) = & (2C_1 R_1 + T)(2C_2 R_2 + T) z^0 \\ & + [(2C_1 R_1 + T)(T - 2C_2 R_2) + (2C_2 R_2 + T)(T - 2C_1 R_1)] z^{-1} \\ & + (T - 2C_1 R_1)(T - 2C_2 R_2) z^{-2}. \end{aligned}$$

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}.$$

$$a_1 = \frac{T - 2C_2R_2}{T + 2C_2R_2} + \frac{T - 2C_1R_1}{T + 2C_1R_1},$$

$$a_2 = \frac{(T - 2C_1R_1)(T - 2C_2R_2)}{(T + 2C_2R_2)(T + 2C_1R_1)}.$$

$$b_0 = T \left(\frac{R_1}{2C_1R_1 + T} + \frac{R_2}{2C_2R_2 + T} \right) + R_0,$$

$$b_1 = \frac{2T^2(R_1 + R_2) + R_0[2T^2 - 8C_1C_2R_1R_2]}{(2C_1R_1 + T)(2C_2R_2 + T)},$$

$$b_2 = \frac{T[R_1(T - 2C_2R_2) + R_2(T - 2C_1R_1)] + R_0(T - 2C_2R_2)(T - 2C_1R_1)}{(2C_1R_1 + T)(2C_2R_2 + T)}.$$

State Space Model Baterai

$$\begin{aligned}\dot{U}_1 &= -\frac{1}{R_1 C_1} U_1 + \frac{1}{C_1} I, \\ \dot{U}_2 &= -\frac{1}{R_2 C_2} U_2 + \frac{1}{C_2} I, \\ \dot{\text{SOC}} &= -\frac{1}{Q_n} I, \\ y &= U_{\text{ocv}}(\text{SOC}) - R_0 I - U_1 - U_2.\end{aligned}$$

Setelah di diskritisasi menggunakan forward euler

$$\begin{aligned}U_{1,k+1} &= U_{1,k} + T_s \left(-\frac{1}{R_1 C_1} U_{1,k} + \frac{1}{C_1} I_k \right), \\ U_{2,k+1} &= U_{2,k} + T_s \left(-\frac{1}{R_2 C_2} U_{2,k} + \frac{1}{C_2} I_k \right), \\ \text{SOC}_{k+1} &= \text{SOC}_k - \frac{T_s}{Q_n} I_k, \\ y_k &= U_{\text{ocv}}(\text{SOC}_k) - R_0 I_k - U_{1,k} - U_{2,k}.\end{aligned}$$

Jacobian dan Hessian untuk UKF

$$F_k = \frac{\partial f}{\partial x} = \begin{bmatrix} 1 - \frac{T_s}{R_1 C_1} & 0 & 0 \\ 0 & 1 - \frac{T_s}{R_2 C_2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Coulomb Counting (untuk referensi)

Algorithm 1 Coulomb Counting

```
1:  $Q_n \leftarrow \text{battery\_capacity}$ 
2:  $SOC[0] \leftarrow \text{initial\_soc}$ 
3: for  $i \leftarrow 1$  to  $N$  do
4:    $dt \leftarrow \text{time}[i] - \text{time}[i - 1]$ 
5:    $I \leftarrow \text{current}[i]$ 
6:    $SOC[i] \leftarrow SOC[i - 1] - \frac{dt \times I}{Q_n}$ 
7:   if  $SOC[i] < 0$  then
8:      $SOC[i] \leftarrow 0$ 
9:   end if
10:  if  $SOC[i] > 1$  then
11:     $SOC[i] \leftarrow 1$ 
12:  end if
13: end for
14: return  $SOC$ 
```

Estimasi SOC

Algorithm 2 Estimasi Parameter dan State

```
1:  $\text{error} \leftarrow \emptyset$ 
2:  $\text{update\_count} \leftarrow 90$ 
3:  $T \leftarrow 5$ 
4:  $\theta, \lambda \leftarrow \text{VFFRLS}(\text{data}[0], \text{error})$ 
5:  $x, \text{error}, \text{params} \leftarrow \text{MIUKF}(\text{data}[0], \theta, T)$ 
6: for  $i \leftarrow 1$  to  $N$  do
7:    $T \leftarrow \text{time}[i] - \text{time}[i - 1]$ 
8:   if  $i \bmod \text{update\_count} = 0$  then
9:      $\theta, \lambda \leftarrow \text{VFFRLS}(x, \text{error\_vec})$ 
10:  end if
11:   $\text{est}, \text{error}, \text{params} \leftarrow \text{MIUKF}(\text{data}[i], \theta, T)$ 
12:   $U_t \leftarrow \text{est}[1]$ 
13:   $SOC \leftarrow \text{est}[3]$ 
14:   $OCV \leftarrow \frac{\theta[1]}{1 + \theta[2] + \theta[3]}$ 
15:   $\text{store}(\theta, \lambda, \text{params}, U_t, SOC, OCV)$ 
16: end for
17: return estimates
```

VFF-RLS

Algorithm 3 Variable Forgetting Factor Recursive Least Squares

Input: x , $error_{UKF}$

```
1: init  $U \leftarrow [0, 0, 0]$  if not initialized
2: init  $I \leftarrow [0, 0, 0]$  if not initialized
3: init  $\theta \leftarrow 0$ ,  $P \leftarrow 100I$  if not initialized
4:  $new\_voltages, new\_currents \leftarrow x$ 
5:  $y \leftarrow newest\_voltages$ 
6:  $\phi \leftarrow [1, new\_voltages, new\_currents]$ 
7:  $L \leftarrow -\rho \times error_{UKF}^2 / S$ 
8:  $\lambda \leftarrow \lambda_{min} + (\lambda_{max} - \lambda_{min}) \times 2^L$ 
9:  $d \leftarrow \lambda + \phi^T P \phi$ 
10:  $K \leftarrow P \phi / d$ 
11:  $P \leftarrow (P - K \phi^T P) / \lambda$ 
12:  $\hat{y} \leftarrow \phi^T \theta$ 
13:  $error \leftarrow y - \hat{y}$ 
14:  $\theta \leftarrow \theta + K e$ 
15: return  $\theta, \lambda, \hat{y}$ 
```

MI-UKF

Algorithm 4 Multi-Innovation Unscented Kalman Filter (MIUKF)

Input: $x = [y; u]$, θ_{out} , T

Output: x_{pred} , $error_MIUKF$, $electParams$

```
1: init  $posterior_x \leftarrow [0; 0; SOC_0]$  if not initialized
2: init  $posterior_P \leftarrow 10^{-3} I_3$  if not initialized
3: init  $error\_history[1:M] \leftarrow 0$  if not initialized
4: init  $gain\_history[1:3, 1:M] \leftarrow 0$  if not initialized
5:  $electParams \leftarrow \text{Levenberg-Marquardt}(\theta_{out}, T)$ 
6: compute UKF weights  $w_x, w_P$  from  $(\alpha, \beta, \kappa, L = 3)$ 
7: form sigma-points  $\Sigma$  around  $posterior_x$  using  $posterior_P$ 
8: for  $i = 1, \dots, |\Sigma|$  do
9:    $\sigma_{pred}[i] \leftarrow \text{battery\_state\_model}(u, \Sigma[i], T, electParams)$ 
10: end for
11:  $a_{priori,x} \leftarrow \sum_i w_x[i] \sigma_{pred}[i]$ 
12:  $a_{priori,P} \leftarrow \sum_i w_P[i] (\sigma_{pred}[i] - a_{priori,x}) (\sigma_{pred}[i] - a_{priori,x})^T + Q$ 
13: for  $i = 1, \dots, |\Sigma|$  do
14:    $y_\sigma[i] \leftarrow \text{battery\_output\_model}(\sigma_{pred}[i], u, electParams, \theta_{out})$ 
15: end for
16:  $y_{pred} \leftarrow \sum_i w_x[i] y_\sigma[i]$ 
17:  $P_{yy} \leftarrow \sum_i w_P[i] (y_\sigma[i] - y_{pred})^2 + R$ 
18:  $P_{xy} \leftarrow \sum_i w_P[i] (\sigma_{pred}[i] - a_{priori,x}) (y_\sigma[i] - y_{pred})$ 
```

MI-UKF (Lanjutan)

```
19:  $K \leftarrow P_{xy} / P_{yy}$ 
20: shift error_history, gain_history right by one
21:  $\text{error\_history}[1] \leftarrow y - y_{\text{pred}}$ 
22:  $\text{gain\_history}[:, 1] \leftarrow K$ 
23:  $\gamma[1] \leftarrow 1, \gamma[2:M] \leftarrow a / (M - 1)$ 
24:  $\text{errorMI} \leftarrow \sum_{i=1}^M \gamma[i] \text{gain\_history}[:, i] \text{error\_history}[i]$ 
25:  $\text{posterior}_x \leftarrow a_{\text{priori}, x} + \text{errorMI}$ 
26: clamp  $\text{posterior}_x[3]$  to  $[0, 1]$ 
27:  $\text{posterior}_P \leftarrow a_{\text{priori}, P} - K P_{yy} K^T$ 
28:  $U_t \leftarrow \text{battery\_output\_model}(\text{posterior}_x, u, \text{electParams}, \theta_{\text{out}})$ 
29:  $x_{\text{pred}} \leftarrow [U_t; u; \text{posterior}_x[3]]$ 
30:  $\text{error\_MIUKF} \leftarrow \text{error\_history}$ 
31: return  $x_{\text{pred}}, \text{error\_MIUKF}, \text{electParams}$ 
```

Model State Baterai

Algorithm 5 Battery State Model

Input: u (current), $x = [U_1; U_2; \text{SOC}]$, T , $\text{electParams} = [R_0; R_1; C_1; R_2; C_2]$

Output: $\text{pred} = [\hat{U}_1; \hat{U}_2; \widehat{\text{SOC}}]$

```
1: Extract parameters:  $R_1, C_1, R_2, C_2 \leftarrow \text{electParams}(2 : 5)$ 
2: Unpack state:  $U_1 \leftarrow x(1), U_2 \leftarrow x(2), \text{SOC} \leftarrow x(3)$ 
3:  $I \leftarrow u$ 
4: Constants:  $Q_n \leftarrow 1.1 \cdot 3600, n \leftarrow 1$ 
5: Time-constants:  $\tau_1 \leftarrow R_1 C_1, \tau_2 \leftarrow R_2 C_2$ 
6: if  $T < \tau_1$  then
7:    $\hat{U}_1 \leftarrow (1 - \frac{T}{\tau_1}) U_1 + \frac{T}{C_1} I$ 
8: else
9:    $\hat{U}_1 \leftarrow R_1 I$ 
10: end if
11: if  $T < \tau_2$  then
12:    $\hat{U}_2 \leftarrow (1 - \frac{T}{\tau_2}) U_2 + \frac{T}{C_2} I$ 
13: else
14:    $\hat{U}_2 \leftarrow R_2 I$ 
15: end if
16:  $\widehat{\text{SOC}} \leftarrow \text{SOC} - \frac{T n}{Q_n} I$ 
17: Clamp  $\widehat{\text{SOC}}$  to  $[0, 1]$ 
18: return  $[\hat{U}_1; \hat{U}_2; \widehat{\text{SOC}}]$ 
```


Model Output Baterai

Algorithm 6 Battery Output Model

Input: $x = [U_1; U_2; *]$, u , $\text{electParams} = [R_0; \dots]$, $\theta_{\text{out}} = [\theta_1; a_1; a_2]$
Output: y (terminal voltage)

- 1: Unpack: $U_1 \leftarrow x(1)$, $U_2 \leftarrow x(2)$, $I \leftarrow u$
- 2: $R_0 \leftarrow \text{electParams}(1)$
- 3: $a_1 \leftarrow \theta_{\text{out}}(2)$, $a_2 \leftarrow \theta_{\text{out}}(3)$
- 4: Compute $\text{OCV} \leftarrow \frac{\theta_{\text{out}}(1)}{1+a_1+a_2}$
- 5: $y \leftarrow \text{OCV} - U_1 - U_2 - I R_0$
- 6: return y

Levenberg-Marquardt

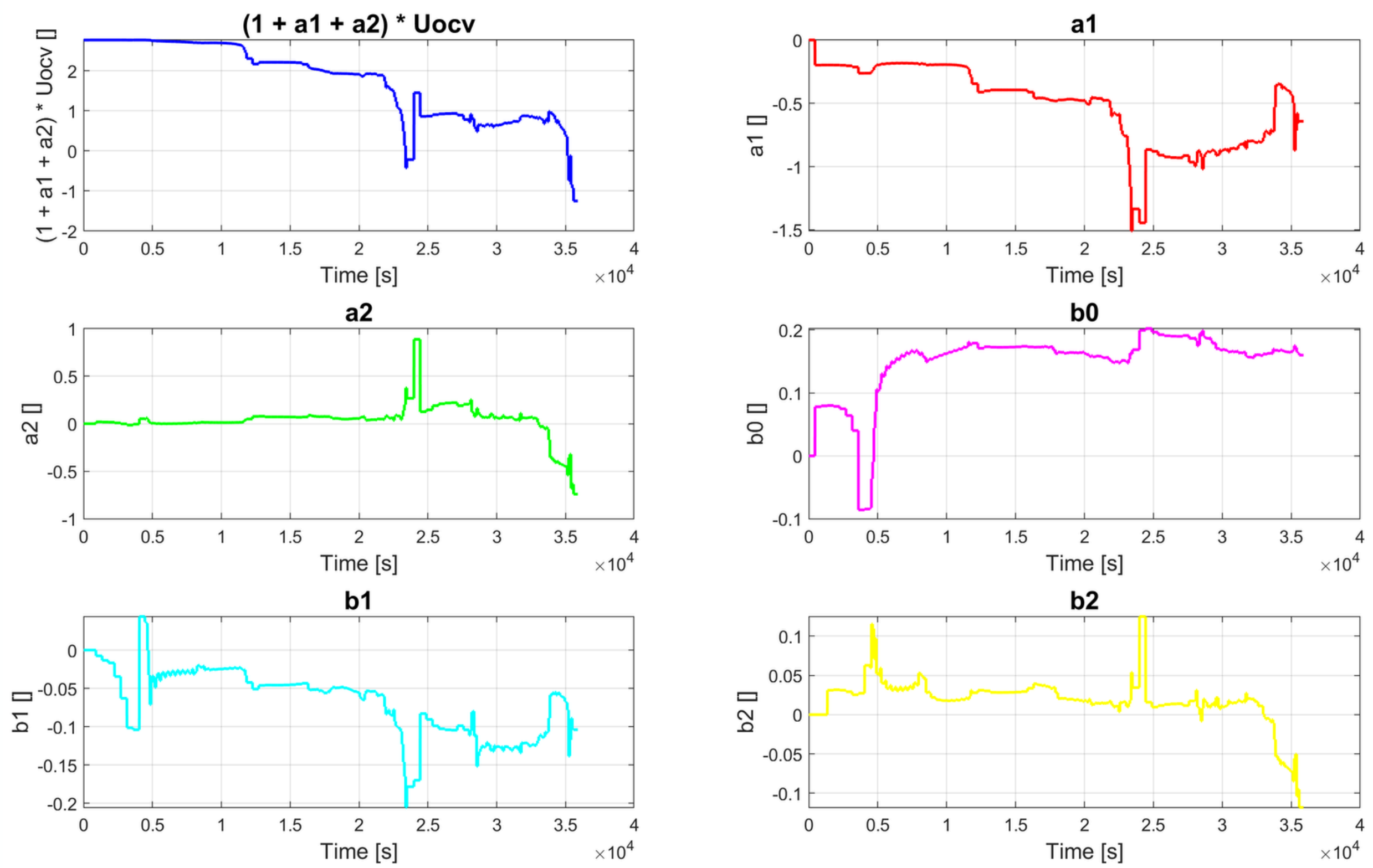
Algorithm 8 Levenberg-Marquardt Electrical-Parameter Estimation

1: **Input:** θ^{out} (ARX coefficients), T (sampling period)
2: **Output:** $x = [R_0, R_1, C_1, R_2, C_2]$

- 3: $a_1^{\text{tgt}} \leftarrow \theta^{\text{out}}[2]$
- 4: $a_2^{\text{tgt}} \leftarrow \theta^{\text{out}}[3]$
- 5: $b_0^{\text{tgt}} \leftarrow \theta^{\text{out}}[4]$
- 6: $b_1^{\text{tgt}} \leftarrow \theta^{\text{out}}[5]$
- 7: $b_2^{\text{tgt}} \leftarrow \theta^{\text{out}}[6]$
- 8: if prev_params not initialized then
- 9: prev_params $\leftarrow [R_{0,0}, R_{1,0}, C_{1,0}, R_{2,0}, C_{2,0}]$
- 10: end if
- 11: $x \leftarrow \text{prev_params}$
- 12: **function** RESIDUALS(x)
- 13: $(R_0, R_1, C_1, R_2, C_2) \leftarrow x$
- 14: $\tau_1 \leftarrow C_1 R_1$, $\tau_2 \leftarrow C_2 R_2$
- 15: $a_1 \leftarrow \frac{T-2\tau_2}{T+2\tau_2} + \frac{T-2\tau_1}{T+2\tau_1}$
- 16: $a_2 \leftarrow \frac{(T-2\tau_1)(T-2\tau_2)}{(T+2\tau_1)(T+2\tau_2)}$
- 17: $b_0 \leftarrow T \left(\frac{R_1}{T+2\tau_1} + \frac{R_2}{T+2\tau_2} \right) + R_0$
- 18: $b_1 \leftarrow \frac{2T^2(R_1+R_2)+R_0(2T^2-8\tau_1\tau_2)}{(T+2\tau_1)(T+2\tau_2)}$
- 19: $b_2 \leftarrow \frac{T[R_1(T-2\tau_2)+R_2(T-2\tau_1)]+R_0(T-2\tau_2)(T-2\tau_1)}{(T+2\tau_1)(T+2\tau_2)}$
- 20: return $[a_1 - a_1^{\text{tgt}}, a_2 - a_2^{\text{tgt}}, b_0 - b_0^{\text{tgt}}, b_1 - b_1^{\text{tgt}}, b_2 - b_2^{\text{tgt}}]^T$
- 21: **end function**
- 22: $x \leftarrow \text{Levenberg-Marquardt}(\text{residuals}, x)$
- 23: prev_params $\leftarrow x$
- 24: return x

Parameter

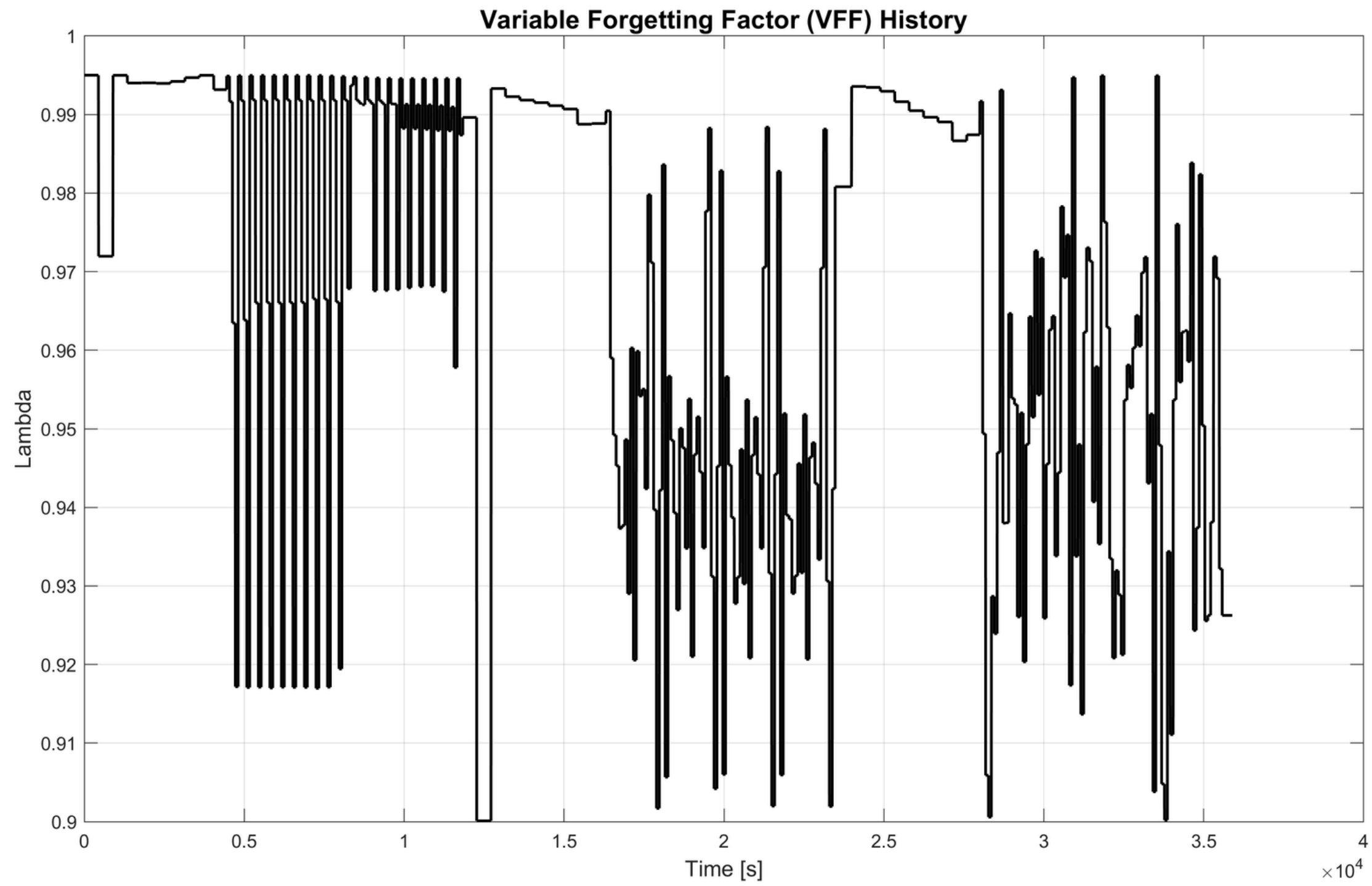
Initialization		MI (Multi-Innovation)	
$x_0 = [U_1, U_2, SOC]^T$	$[0, 0, (2.8-2.7)/(3.6-2.7)] = [0, 0, \approx 0.1111]^T$	M (innovation length)	22
$\theta_0 = [R_0, R_1, C_1, R_2, C_2]^T$	$[0, 0, 0, 0, 0]^T$	a (weighting coeff.)	0.5
P_0 (param covariance)	$1 \times 10^{-3} \cdot I_{3 \times 3}$ MI-UKF; $1 \times 10^2 \cdot I_{6 \times 6}$ VFFRLS		
VFFRLS		UKF	
λ_{min}	0.9	L	3
λ_{max}	0.995	α	1×10^{-3}
ρ (sensitivity)	200	k_i	0
S (window length)	22	β	2
$T\theta$ (update interval)	90 samples	Q_k (noise proses)	$1 \times 10^{-8} \cdot I_{3 \times 3}$
		R_k (noise pengukuran)	1×10^{-8}



Terlihat bahwa nilai dari setiap parameter diskrit mengalami fluktuasi selama berjalannya program dan mengalami beberapa perubahan nilai signifikan yang terlihat dalam bentuk lonjakan nilai.

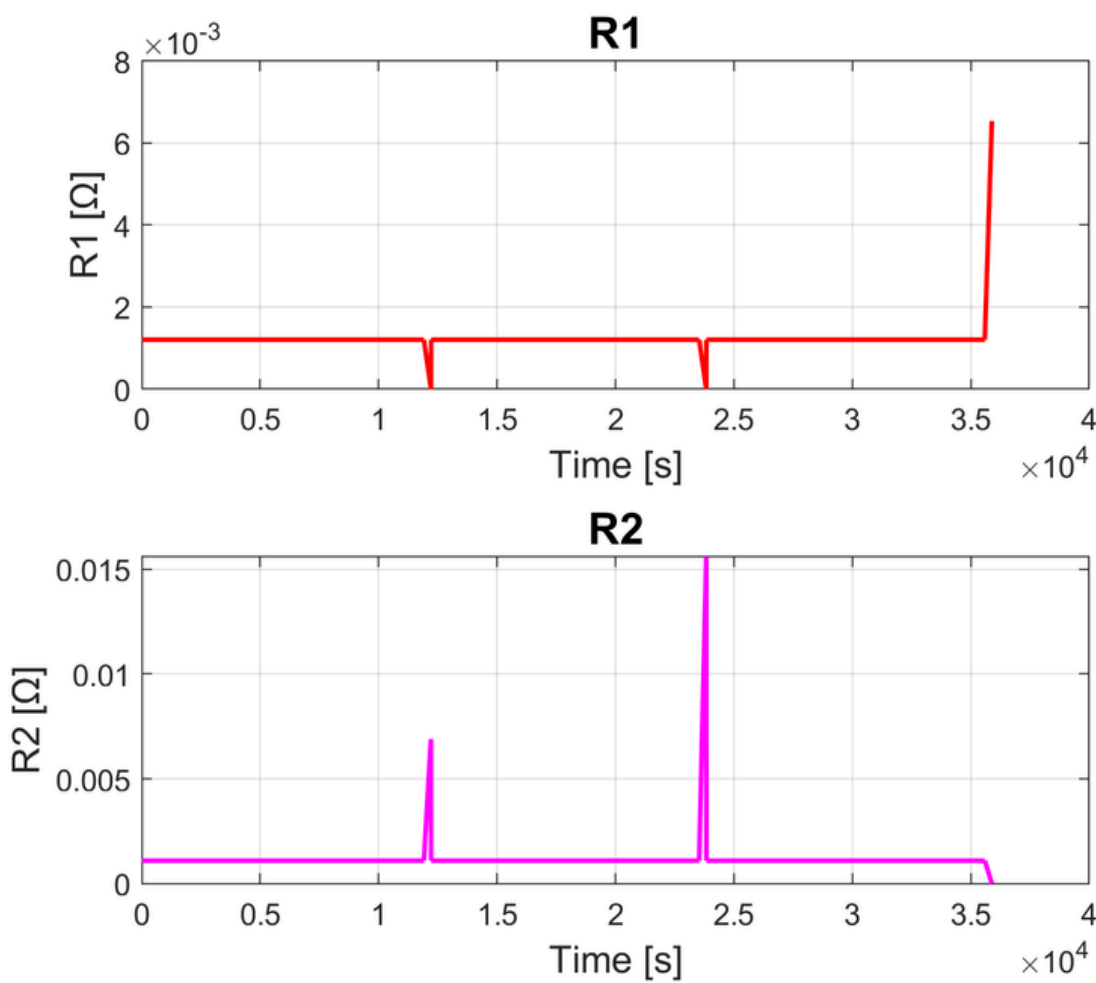
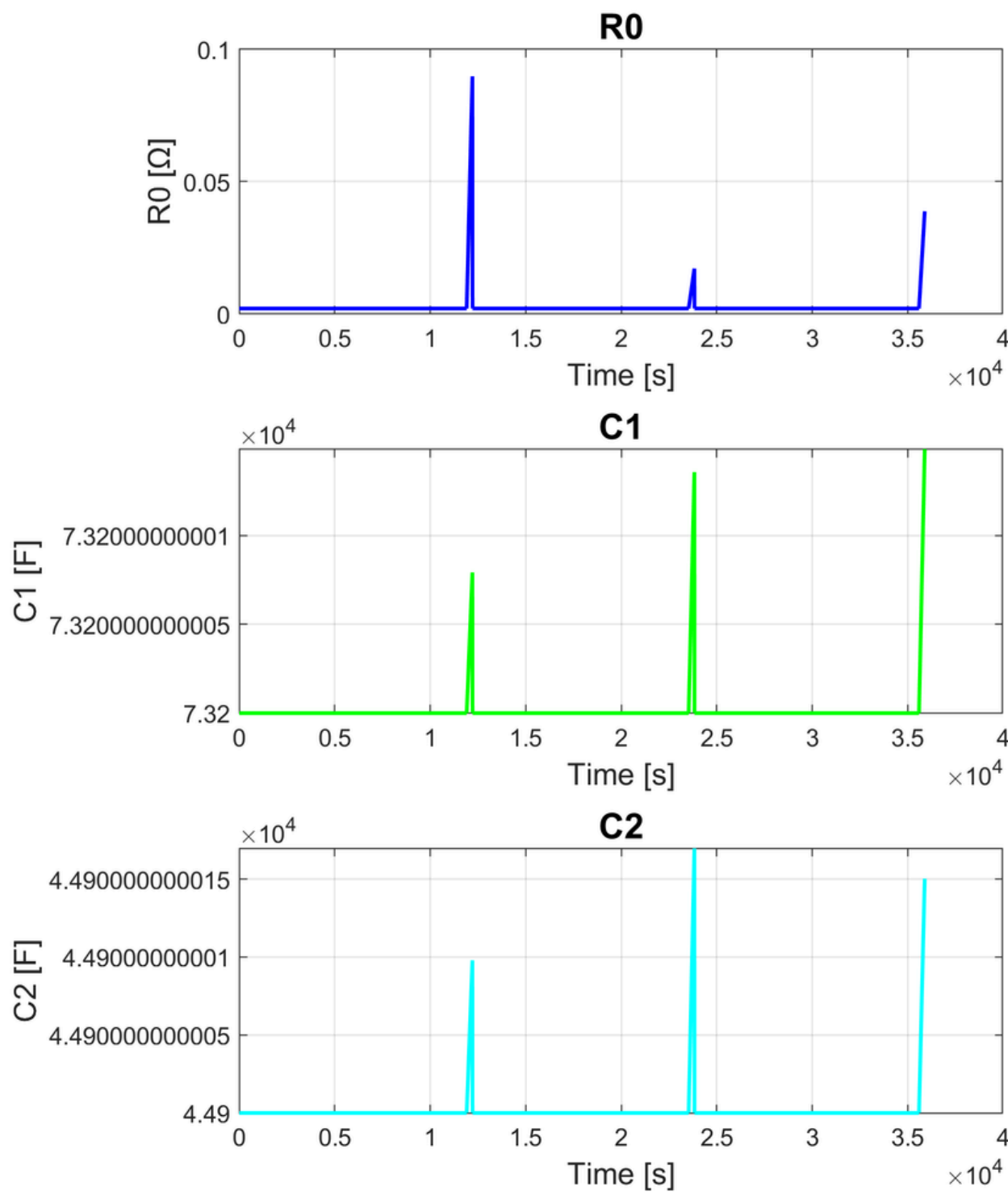
Lonjakan nilai pertama terjadi saat transisi charging->DST discharging dan kedua adalah US06 discharging → charging. Terlihat masa transisi menyebabkan ketidakstabilan dalam prediksi sistem.

Grafik nilai bobot *forgetting factor* pada tiap pencuplikan



Terlihat perubahan nilai lambda atau *forgetting factor* selama iterasi terjadi cukup melambangkan fase charging yg mulus dan fase discharge saat dilakukan *dynamic test profile*. Nilai lambda banyak berubah saat discharge karena nilai tegangan dan arus selama discharge yang terekam memiliki fluktuasi yang mempengaruhi adaptabilitas algoritma VFFRLS-MIUKF untuk mengidentifikasi sistem.

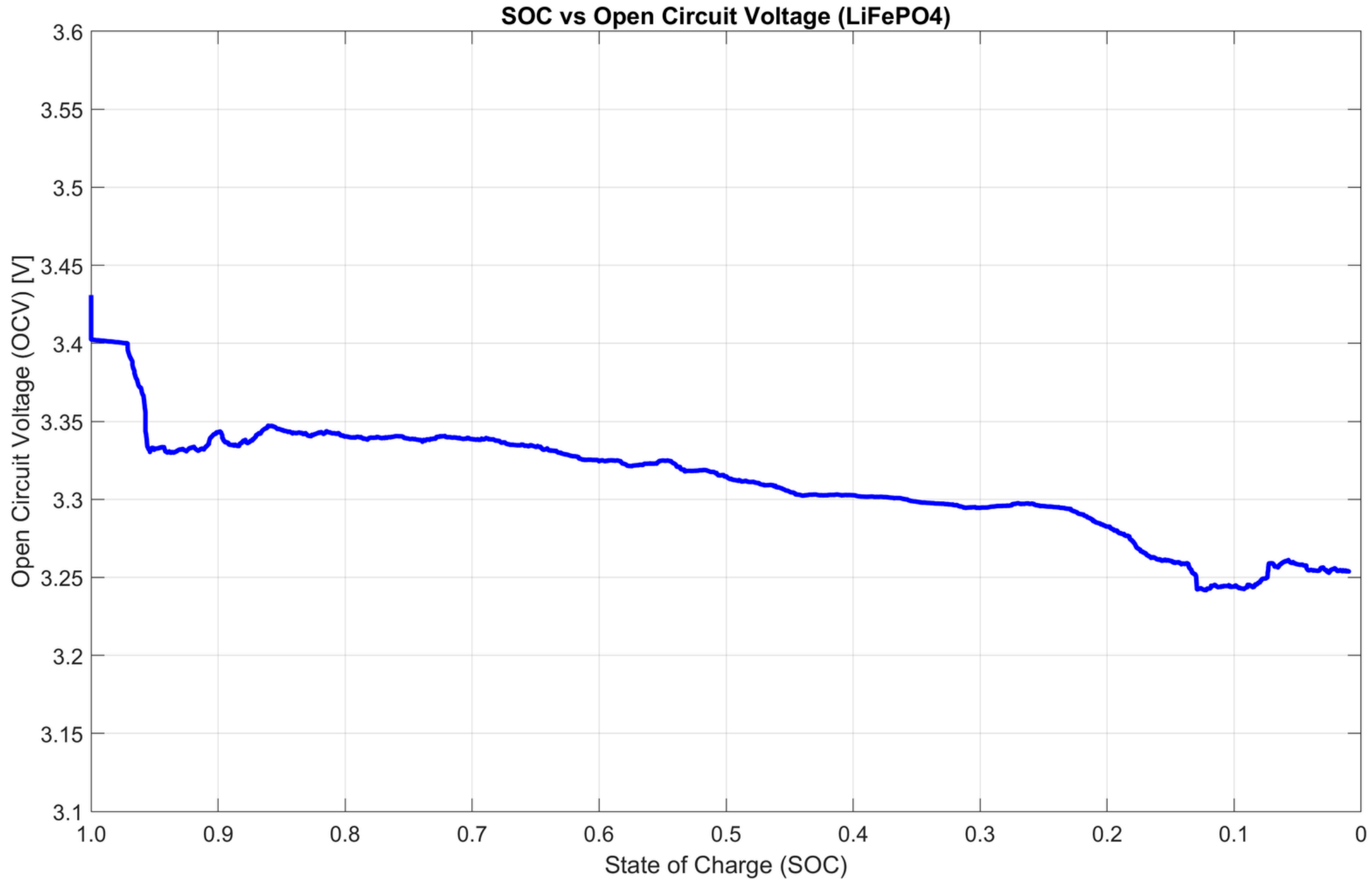
Grafik nilai parameter model elektrik pada tiap pencuplikan



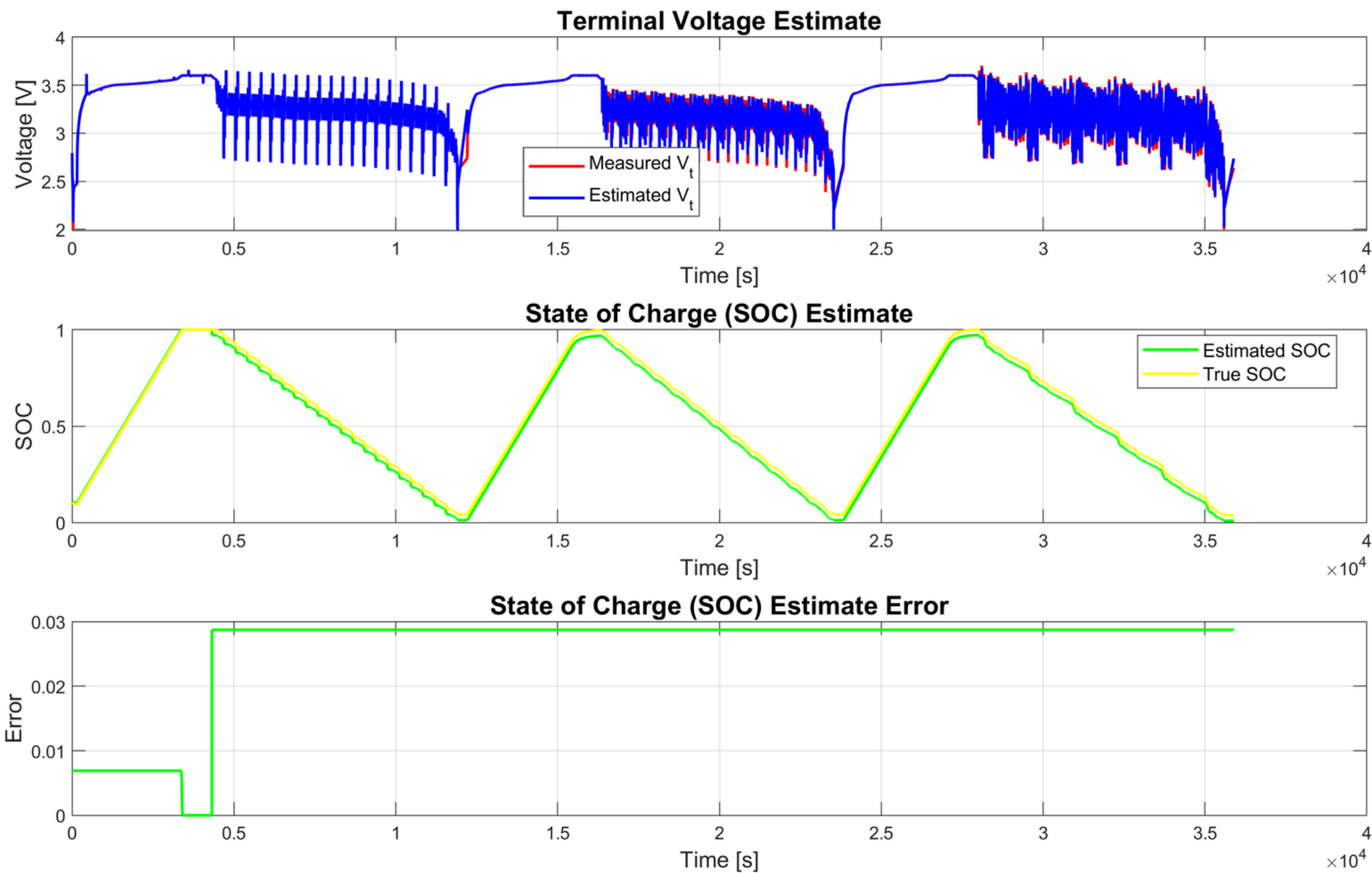
Menggunakan fungsi Isqnonlin pada MATLAB untuk menerapkan algoritma Levenberg-Marquadt didapatkan nilai parameter elektrik optimal dengan nilai batas bawah 0. Terlihat pada fase charge dan discharge nilai relatif konstan, namun pada saat terjadi transisi dari discharge kembali ke fase charging pada seluruh variabel terdapat lonjakan nilai yang sangat signifikan.

Walaupun begitu, selama program berjalan Levenberg-Marquadt tetap konvergen dan menemukan nilai parameter optimal.

Grafik karakteristik baterai Voc vs SOC



Grafik SOC - OCV hasil dari estimasi nilai OCV oleh VFFRLS menunjukkan gradien yang cukup datar dari nilai 0.9 sampai 0.1 seperti yang dapat diekspektasi dari baterai LiFePO4. Walaupun begitu, dikarenakan memanfaatkan estimasi dari VFFRLS yang dijalankan tidak setiap iterasi (mengikuti jurnal) dan data yang berfluktuasi menghasilkan grafik yang kurang sempurna karena kurang sempurnanya prediksi Uocv oleh RLS.



Melalui hasil parameter dari VFFRLS, MIUKF berhasil memberikan output berupa estimasi U_t dari pemodelan baterai serta SOC dari *coulomb counting*. Terlihat saat charging SOC meningkat dan saat discharging SOC menurun. Estimasi SOC serta U_t juga cukup mendekati dari nilai aslinya dengan estimasi SOC hanya mengalami error maksimal sebesar 0.03 dari rasio 1.

Ketiga tipe *dynamic profile test* juga dapat terlihat dari nilai tegangan yang mana masing-masing menghasilkan fluktuasi dengan ciri berbeda namun berkat VFFRLS dan MIUKF perhitungan estimasi mampu mengikutinya dengan cukup baik.

Model VFFRLS dengan MIUKF berhasil untuk memodelkan baterai dan menghasilkan nilai prediksi V_t serta SOC yang cukup akurat dengan pendekatan diskritisasi metode Tustin pada model ARX untuk menghubungkan input-output sistem serta diskritisasi Forward Euler untuk memodelkan state space baterai.

Nilai Lambda (Forgetting Factor) selama iterasi berubah-ubah untuk meningkatkan / menurunkan fleksibilitas sistem sesuai dengan kondisi yang dibutuhkan.

Levenberg-Marquadt berhasil mengoptimasi 5 persamaan non linear yang menghubungkan parameter model diskrit a_1, a_2, b_0, b_1, b_2 terhadap parameter model elektrik R_0, R_1, C_1, R_2, C_2 .

Parameter Elektrik memiliki kecenderungan bernilai statis selama fase discharge/ SOC cukup tinggi namun saat SOC rendah / transisi discharge ke charging pada data yang digunakan memiliki kecenderungan untuk memiliki nilai yang meningkat drastis untuk memodelkan baterai dengan SOC rendah.

Terima Kasih